Programare Orientata pe Obiecte, 26 ianuarie 2019

1. Care dintre urmatorii sunt specificatori de acces valizi in Java?
   a. public
   b. void
   c. private
   d. final
2. In codul problemei sunt definite:
   a. 1 clasa abstracta si 2 clase obisnuite
   b. 1 clasa abstracta, 2 clase obisnuite si 1 clasa interna anonima
   c. 1 clasa abstracta si 3 clase obisnuite
   d. 1 clasa abstracta, 3 clase obisnuite si 1 clasa interna anonima
3. Care dintre urmatoarele afirmatii este corecta?
   a. Compunerea este o relatie mai puternica decat Agregarea
   b. Agregarea este o relatie de tip HAS-A
   c. Asocierea este o relatie mai puternica decat Agregarea
   d. Mostenirea este o relatie de tip HAS-A
4. Relatia dintre clasele Animal si String este de:
   a. Asociere
   b. Agregare
   c. Compunere
   d. Mostenire
5. O subclasa aflata in acelasi pachet cu superclasa mosteneste:
   a. toate campurile si metodele public
   b. toate campurile si metodele private
   c. toate campurile si metodele protected
   d. toate campurile si metodele default
6. In clasele Dog si Cat, in constructor, este necesara apelarea constructorului din Animal cu super?
   a. Nu, deoarece clasa Animal este abstracta
   b. Da, deoarece este o mostenire multipla
   c. Da, deoarece clasa Animal nu are niciun constructor default
   d. Nu, deoarece clasa Animal are un singur constructor
7. In ce conditii o clasa obisnuita poate extinde o clasa abstracta:
   a. Daca nu are nicio metoda statica
   b. Daca sunt in acelasi pachet
   c. Daca ii implementeaza toate metodele abstracte
   d. Daca suprascrie cel putin o metoda a clasei abstracte.
8. Daca am dori sa adaugam clasa Bulldog, aceasta ar avea sens:
   a. Sa extinda clasa Animal
   b. Sa fie compusa in clasa Animal
   c. Sa extinda clasa Dog
   d. Sa fie compusa in clasa Dog
9. Problema diamant apare atunci cand doua clase B si C mostenesc o clasa A, si o alta clasa D mosteneste atat B, cat si C, intrucat:
   a. Daca D suprascrie o metoda din B, pe care C nu o are, rezulta un comportament nedeterminat
   b. Daca o metoda in A e suprascrisa de B si C, fara ca D sa o suprascrie, este ambiguu pe care implementare o va mosteni
   c. Daca B si C trebuie sa suprascrie o metoda din A in acelasi fel se duplica cod
   d. Nu putem face ca C si D sa suprascrie o metoda din A, dar B nu.
10. Ce va intoarce functia compare pentru obiectele obtinute din urmatorul input:

```
Cat Cheshire 5
Dog Lassie 7
```

    a. 2 b. 1 c. -1 d. -2
11. La curs am discutat despre polimorfism:
    a. Static
    b. Dinamic
    c. Complex
    d. Parametric
12. Faptul ca in cod se apeleaza print pentru referinte de tip Animal, care de fapt se refera la obiecte Dog si Cat este o folosire a polimorfismului:
    a. Static
    b. Dinamic
    c. Complex
    d. Parametric
13. O clasa interna anonima:
    a. Poate sa extinda o singura clasa sau sa implementeze o singura interfata
    b. Nu are constructori proprii
    c. Este instantiata intr-un singur loc
    d. Trebuie sa extinda java.lang.Anonymous
14. Care dintre urmatoarele inputuri va determina programul sa arunce exceptie la rulare:
    a. CatJohn 5
    b. Mouse Mickey 120
    c. Dog Wolfy 12.5
    d. Cat 3.14 10000
15. Care dintre urmatoarele este superclasa tuturor erorilor si exceltiilor din Java?
    a. RunTimeException
    b. Throwable
    c. Catchable
    d. Exception
16. LSP este reprezentat in codul de la problema prin faptul ca:

Scanned by CamScanner

a. Clasele Dog si Cat difera doar prin metoda getType()
b. Se foloseste metoda sort si interfata Comparator pentru a realiza sortarea
c. In printList toti membrii listei sunt tratati la fel indiferent de implementare
d. Metoda printList este statica

17. Care principiu de design orientat obiect militeaza ca o clasa nu ar trebui fortata sa implementeze metode pe care nu le va folosi?
a. SRP
b. LSP
c. ISP
d. DIP

18. Care principiu de design orientat obiect este incalcat de clasa Solution prin faptul ca se ocupa de citire, sortare si afisare?
a. SRP
b. LSP
c. ISP
d. DIP

19. DIP este util intrucat, atunci cand este respectat:
a. implementarile pot fi inlocuite fara a afecta codul care le foloseste
b. o clasa nu este fortata sa implementeze metode care nu vor fi folosite
c. obiectele dintr-un program pot fi inlocuite cu implementari ale subtipurilor
d. reduce numarul de clase si interfete din program

20. Dat fiind codul de la problema, as fi putut sa pun toate instantele de Cat si Dog intr-o singura List<Animal> si sa le sortez?
a. Nu, nu pot compara caini si pisici
b. Nu, Animal este o clasa abstracta
c. Da, pot compara caini si pisici
d. Da, Animal mosteneste implementarea functiei compareTo de la Object

21. Prin Type Erasure:
a. Tipul formal este substituit cu Object daca nu este upper bounded
b. La compilare se produce un singur cod pentru o clasa generica
c. Tipul formal este substituit cu lower bound daca este lower bounded
d. Tipul formal este substituit cu upper bound daca este upper bounded

22. Ce design pattern este folosit de clasa Scanner?
a. Visitor
b. Iterator
c. Observer
d. Builder

23. Care dintre urmatoarele se incadreaza in categoria de Creational Design Patterns:
a. Visitor
b. Observer
c. Singleton
d. Factory

24. Faptul ca sort apeleaza compare este o exemplificare a carui design pattern?
a. Visitor
b. Strategy
c. Observer
d. Factory

25. In care pattern dintre urmatoarele nu se poate aplica DIP?
a. Observer
b. Factory
c. Singleton
d. Visitor

26. Ce design pattern ar putea fi folosit pentru a crea obiecte din ierarhia clasei Animal in contextul problemei:
a. Singleton
b. Factory
c. Builder
d. Strategy

27. Care dintre urmatoarele tehnici poate fi aplicata pentru a pacali o implementare de Singleton bazata pe constructor privat?
a. Mostenire
b. Introspectie
c. Reflectie
d. Adnotare

28. Ce efect are urmatorul cod introdus in main dupa ce se termina bucla while?
```
Method method =
catList.getClass().getMethod("clear");
method.invoke(catList);
```
a. Exceptie la runtime
b. Pisicile nu vor fi sortate
c. Nu vor fi afisate pisici
d. Niciun efect

PROBLEMA:

1. Explicati de ce declararea functiei printList sub forma `printList(List<Animal> animals)` genereaza eroare la compilare si de ce forma din cod este cea corecta.
2. Comentati cu privire la extensibilitatea programului. Ce fel de modificari ar putea fi dorite si cat de usor sau greu ar putea fi implementate?

Scanned by CamScanner

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

abstract class Animal {
    String name;
    Integer age;

    public Animal(String name, int age) {
        this.age = age;
        this.name = name;
    }

    void print() {
        System.out.println("Type: " + getType() + " Name: " +
                           name + " Age: " + age);
    }

    abstract String getType();
}

class Dog extends Animal {
    public Dog(String name, int age) {
        super(name, age);
    }

    @Override
    String getType() {
        return "Dog";
    }
}

class Cat extends Animal {
    public Cat(String name, int age) {
        super(name, age);
    }

    @Override
    String getType() {
        return "Cat";
    }
}
```

```
Public class Solution {

    //TODO 1: Instatiate the comparator to sort elements
    // by age first (ascending) name second (alphabetically)
    static Comparator<Animal> comp = new Comparator<Animal>() {
        @Override
        public int compare(Animal o1, Animal o2) {
            if (o1.age.equals(o2.age)) {
                return o1.name.compareTo(o2.name);
            } else {
                return (int) (o1.age - o2.age);
            }
        }
    };

    //TODO 2: Implement the printList method so that it receives
    // a list of instances of a class that extends Animal, and
    // calls print() on all of those instances
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<Cat> catList = new ArrayList<>();
        List<Dog> dogList = new ArrayList<>();
        while (scanner.hasNext()) {
            String line = scanner.nextLine();

            String[] splittedLine = line.split(" ");
            String type = splittedLine[0];
            String name = splittedLine[1];
            int age = Integer.parseInt(splittedLine[2]);

            if (type.equals("Cat")) {
                catList.add(new Cat(name, age));
            } else if (type.equals("Dog")) {
                dogList.add(new Dog(name, age));
            }
        }
        catList.sort(comparator);
        dogList.sort(comparator);
        printList(catList);
        printList(dogList);
    }

    static void printList(List<? extends Animal> animals) {
        for (Animal animal : animals) {
            animal.print();
        }
    }
}
```

Scanned by CamScanner